

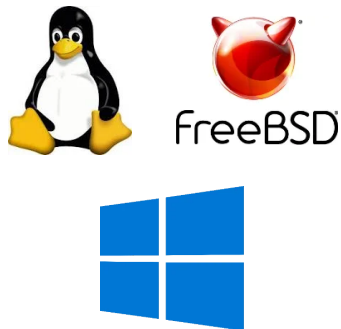
Beyond Zero-Cost

Understanding Rust Safety Overheads in Systems Code

Soham Bagchi · Manvik Nanda · Anton Burtsev

PLOS '26

Kernels



Cloud infrastructure



Latency-sensitive services



Prior claims

Bounds checking accounts for 52.7% of Rust's overhead on algorithm benchmarks

Zhang et al. 2022

Many more instructions per packet than C, only a small throughput loss

ixy · Emmerich et al. 2019

Near-C for simple network functions, but a 14% slowdown on longest-prefix match

NetBricks · Panda et al. 2016

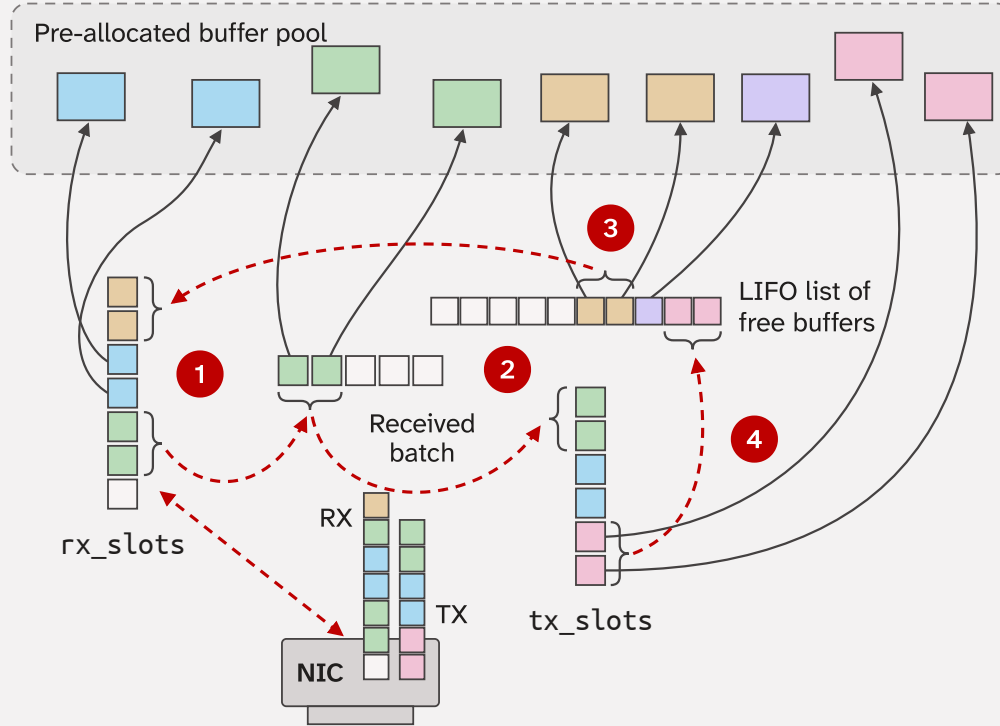
Careful code structure eliminates many runtime checks — Rust can be fast

Rust Performance Book · Nethercote 2020

Our question

How much overhead does Rust introduce in the critical path of a high-throughput device driver?

Driver Architecture





Microarchitectural Analysis



```
void tx_enqueue(struct driver *d,
                uint16_t idx, struct pkt_buf *buf) {
    ...
    struct pkt_buf *old_buf = d->tx_slots[idx];
    d->tx_slots[idx] = buf;
    d->free -= 1;
    ...
}
```

```
mov rax,QWORD PTR [rsp+0xe30] ; base
mov r15,QWORD PTR [rax+rdx*8] ; read old
mov QWORD PTR [rax+rdx*8],rbx ; write new
dec WORD PTR [rsp+0xe80] ; free -= 1
```

Prevented at

COMPILE TIME

Use-after-free

Double-enqueue

RUNTIME

Null dereference

Out-of-bounds

Integer wrapping

Panic aborts



```
fn tx_enqueue(&mut self, idx: u16, buf: TxBuf) {
    ...
    let old_buf = mem::replace(
        &mut self.tx_slots[idx as usize],
        Some(buf.into_in_flight()))
        .expect("missing tx parked slot");
    self.free -= 1;
    ...
}
```

+ mov rsi,QWORD PTR [rsp+0xe48]	; load len
+ cmp rsi,rdi	; idx < len?
+ jbe 1fac2	; panic
mov rax,QWORD PTR [rsp+0xe40]	; reload base
mov r13,QWORD PTR [rax+rdi*8]	; read old
mov QWORD PTR [rax+rdi*8],rbx	; write new
+ test r13,r13	; None?
+ je 1face	; panic
dec WORD PTR [rsp+0xe80]	; free -= 1

Prevented at

COMPILE TIME

✓ Use-after-free

✓ Double-enqueue

RUNTIME

✓ Null dereference

✓ Out-of-bounds

Integer wrapping

Panic aborts



```
fn tx_enqueue(&mut self, idx: u16, buf: TxBuf) {
    ...
    let old_buf = mem::replace(
        &mut self.tx_slots[idx as usize],
        Some(buf.into_in_flight()))
        .expect("missing tx parked slot");
    self.free -= 1;
    ...
}
```

```
# Cargo.toml - [profile.release]
- overflow-checks = false
+ overflow-checks = true
```

Prevented at

COMPILE TIME

✓ Use-after-free

✓ Double-enqueue

RUNTIME

✓ Null dereference

✓ Out-of-bounds

✓ Integer wrapping

Panic aborts

```
mov rsi,QWORD PTR [rsp+0xe48] ; load len
cmp rsi,rdi ; idx < len?
jbe 1fac2 ; panic
mov rax,QWORD PTR [rsp+0xe40] ; reload base
mov r13,QWORD PTR [rax+rdi*8] ; read old
mov QWORD PTR [rax+rdi*8],rbx ; write new
test r13,r13 ; None?
je 1face ; panic
- dec WORD PTR [rsp+0xe90] ; free -= 1
+ movzx eax,WORD PTR [rsp+0xe90] ; load free
+ test ax,ax ; free == 0?
+ je 2205d ; panic
+ dec eax ; free - 1
+ mov WORD PTR [rsp+0xe90],ax ; store
```



```
fn tx_enqueue(&mut self, idx: u16, buf: TxBuf)
-> Result<(), DriverError> {
    ...
    let slot = self.tx_slots
        .get_mut(idx as usize)
        .ok_or(DriverError::ReflectIndexOutOfRange {
            what: "tx_slots", index: idx as usize })?;
    let old_buf = mem::replace(
        slot, Some(buf.into_in_flight()))
        .ok_or(DriverError::MissingTxInFlightBuf {
            index: idx })?;
    self.free -= 1;
    ...
}
```

Prevented at

COMPILE TIME

✓ Use-after-free

✓ Double-enqueue

RUNTIME

✓ Null dereference

✓ Out-of-bounds

Integer wrapping

✓ Panic aborts

```
cmp QWORD PTR [rsp+0xe18],r8 ; bounds
jbe 1c492 ; → error
mov rax,QWORD PTR [rsp+0xe10] ; swap
mov r14,QWORD PTR [rax+r8*8]
mov QWORD PTR [rax+r8*8],r12
+ mov WORD PTR [rsp+0x452],di ; build err
+ mov WORD PTR [rsp+0x450],0x13
test r14,r14 ; None?
je 1c4b2 ; → error
+ mov DWORD PTR [rsp+0x248],edi ; spill
+ lea rdi,[rsp+0x450] ; drop err
+ call drop_in_place<DriverError>
dec WORD PTR [rsp+0xe80] ; free -= 1
```

```
drop_in_place<DriverError>: ; 11 instrs
push r15 / push r14 / push rbx ; prologue
movzx eax,WORD PTR [rdi] ; load tag
lea ecx,[rax-0x1] / cmp ecx,0x17
jae 69736 ; not taken
pop rbx / pop r14 / pop r15 / ret; epilogue
```



```
fn tx_enqueue(&mut self, idx: u16, buf: TxBuf) {
    ...
    let slot = unsafe {
        self.tx_slots.get_unchecked_mut(idx as usize)
    };
    let old_buf = unsafe {
        mem::replace(slot, Some(buf.into_in_flight()))
            .unwrap_unchecked()
    };
    self.free -= 1;
    ...
}
```

```
= mov rax,QWORD PTR [rsp+0xe30] ; base
= mov r15,QWORD PTR [rax+rdx*8] ; read old
= mov QWORD PTR [rax+rdx*8],rbx ; write new
= dec WORD PTR [rsp+0xe80] ; free -= 1
```

Prevented at

COMPILE TIME

✓ Use-after-free

✓ Double-enqueue

RUNTIME

Null dereference

Out-of-bounds

Integer wrapping

Panic aborts

Identical to C in this window. Across the critical path: 112 instructions vs 113, +3 stores, 96% of C's throughput.

Two Ways to Name a Buffer

Buffer Pool

The handle is the address

&mut PktBuf



buffer

```
mov rbx,QWORD PTR [r12+rcx*8] ; handle = addr
mov QWORD PTR [rbx+0x10],rdx ; use buffer
```

Ixy-Like

Shift and add, twice per packet

idx: u32



shl · add base



buffer

```
mov ebx,DWORD PTR [r12+rcx*4] ; load idx
+ shl rbx,0xb ; idx × 2048
+ add rbx,QWORD PTR [rsp+0x38] ; + pool base
mov QWORD PTR [rbx+0x10],rdx ; use buffer
```

Performance Analysis

4 C variants

6 Rust variants

BUFFER POOL

C|RefPool

C|Safe(RefPool)

C|Safe(RefPool+Overflow)

Rust|Unsafe(RefPool)

Rust|Safe(RefPool)

Rust|Safe(RefPool+Overflow)

Rust|Safe(RefPool+Errors)

IXY-LIKE

C|PtrArith

Rust|Unsafe(PtrArith)

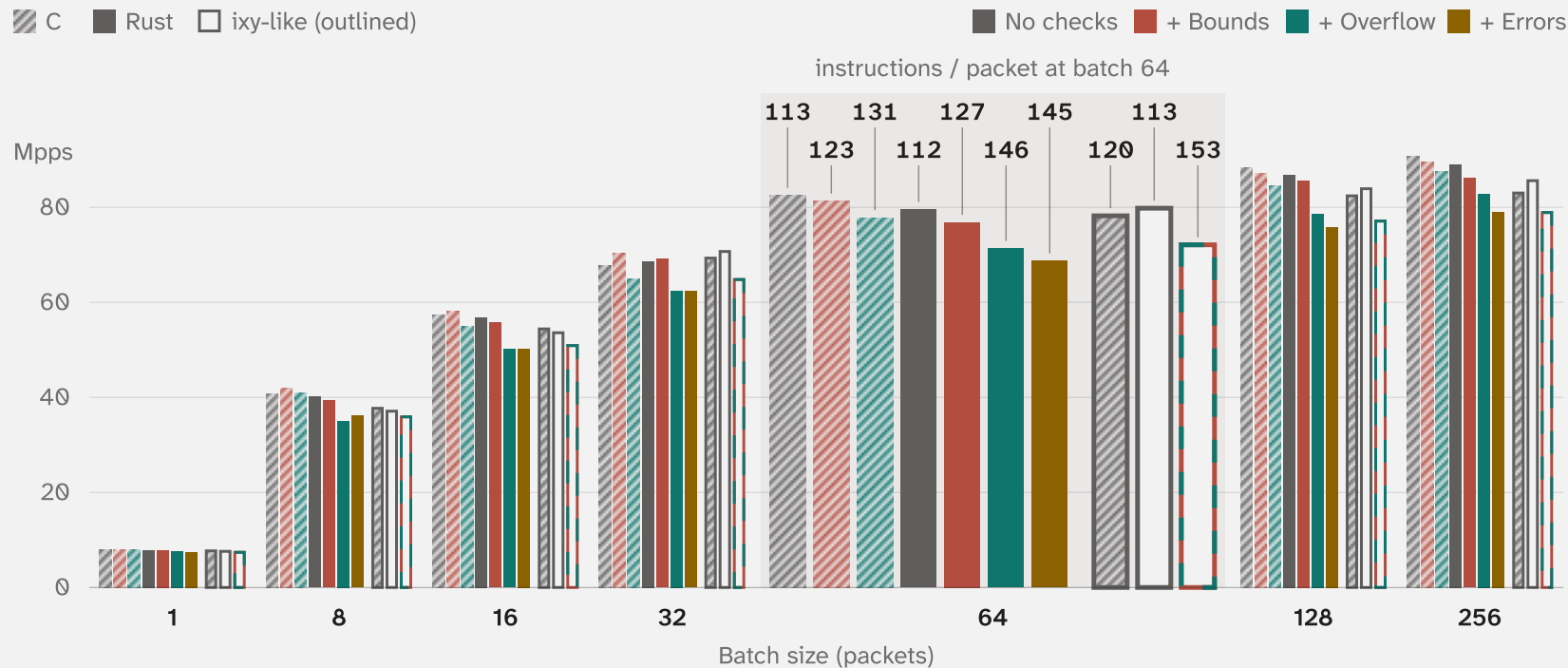
Rust|Safe(PtrArith+Overflow)

TESTBED

Xeon Gold 5512U @ 3.0 GHz · Intel E810-C 100 GbE

64B frames · L2 forwarding · one RX/TX pair

Compared: Throughput and Instruction Count



Predicting Per-Packet Costs

$$\text{cyc/pkt} = 20.7 + 0.063 I + 0.283 S + 2.33 X$$

I instructions · S stores · $X = 1$ for PtrArith

0.063

cycles per instruction

Instruction

≈ 0.10

cycles per load

Load

≈ 0.35

cycles per store

Store

Takeaways (for us all)

93%

of unsafe C's throughput achievable from safe Rust

Feature	Instructions / pkt	Throughput
Bounds + null checks	+15	-3.3%
Overflow checks	+19	-7.2%
Error returns	+18	-10.5%

FOR DEVELOPERS

Write lean Rust that follows C idioms

Keep representations compact to preserve niche optimisation

TO CLOSE THE GAP

Compilers Inline `Result<T, E>` - returning functions

Hardware More store-buffer and register capacity

Thank you

github.com/mars-research/userspace-ice

doi.org/10.1145/3831586.3838147